
Chapter 5

Decision trees

Assoc. Prof. Dr. Duong Tuan Anh
Faculty of Computer Science and Engineering,
HCMC Univ. of Technology
3/2015

Outline

- ❑ Introduction to decision tree
- ❑ Decision tree for pattern recognition
- ❑ Construction of decision trees
- ❑ Splitting at the nodes
- ❑ Overfitting and pruning
- ❑ Example of decision tree induction

1. Introduction

- A *decision tree* is a tree where each non-leaf node is associated with a *decision* and the leaf nodes are associated with an *outcome* or *class label*.
- Each internal node test one or more attribute values leading to two or more branches. Each *branch* in turn is associated with a *possible value of the decision*. These branches are mutually distinct and collective exhaustive.
- Decision trees are excellent tools for choosing between several courses of action. They provide a highly effective structure with which you can lay out options and investigate the possible outcome of choosing these options.
- In the case of *binary decision tree*, there are two outgoing edges from the nodes. One edge represent the outcome “yes” or “true” and the other edge, the outcome “no” or “false”.
- Decision tree classifier: ***inductive learning***

2. Decision Trees for Pattern Recognition

Patterns can be classified using *decision trees* where the nodes in the tree represent the status of the problem after making some decision.

The *leaf nodes* give the *class label* of the classification rule corresponding to the *path* from root node to the leaf node.

Example: Consider employee records in a company as in the following table:

Name	Age	Qualification	Position

Ram	55	B. Com.	Manager
Shyam	30	B. Eng.	Manager
Mohan	40	M.Sc.	Manager

Given the set of patterns as in the following table:

Manager	No. of Assistants	Mood	Output
Shyam	3	No	Medium
Shyam	5	No	Medium
Shyam	1	Yes	High
Ram	1	Yes	Low
Ram	5	No	Low
Ram	5	Yes	Low
Mohan	1	No	Low
Mohan	3	Yes	Medium
Mohan	5	No	High

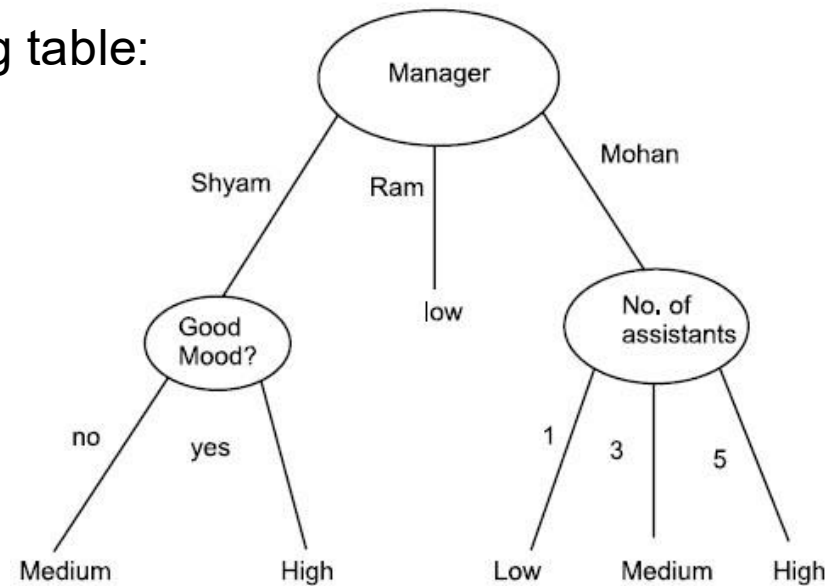


Figure 5.1 Decision tree for pattern classification

The decision tree is induced from the patterns in the table.

The output of these managers is the class labels (high, medium and low). “Manager” is a categorical attributes, “no. of assistants” is a numerical attribute” and “mood” is a boolean attribute.

Notes:

- Class labels are associated with leaf nodes
- Root to leaf represents a rule
 - *If (Manager = Mohan” and “No. of assistants = 3) then (Output = medium)*
- Every node involves making a decision
- Irrelevant features
 - If each pattern has a feature (the age of manager), this feature does not appear in the decision tree.
- Numerical and categorical features
 - Both numerical and categorical features appear in the decision tree.
- The rules are simple and easy to understand

Weakness of decision trees

- Design time could be large
 - For example, the number of oblique splits possible could be exponential in the number of training patterns
- Simple (axis-parallel) decision trees do not represent *non-rectangular regions* well.
 - Most decision tree algorithms only examine a single attribute at a time. This leads to rectangular classification that may not correspond well with the actual distribution of the data.
 - In the simple decision tree, all the decision boundaries are hyperplanes orthogonal to the axis of the feature being considered. When the regions are non-rectangular, the decision tree approximates the regions by *hyper-rectangle*.

Example of non-rectangular decision boundary

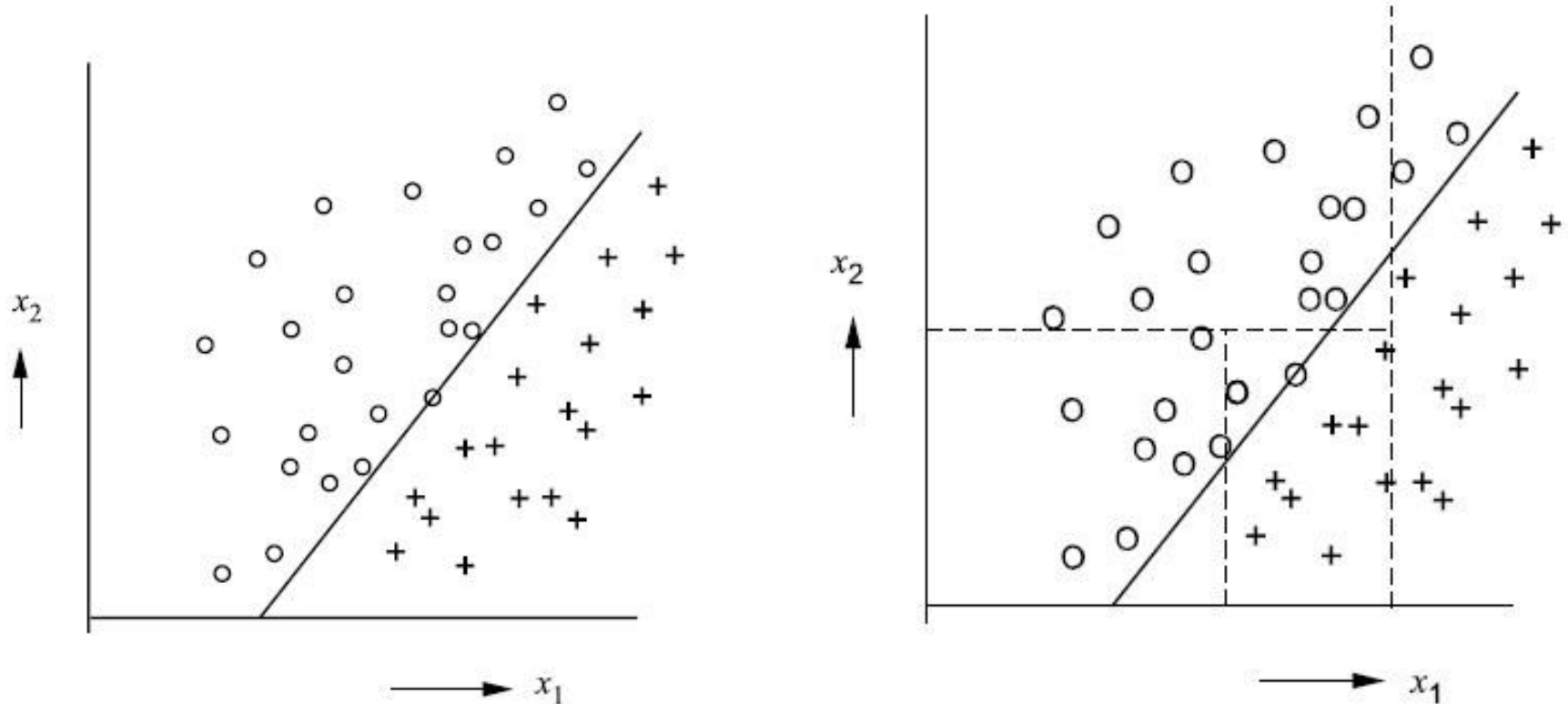


Figure 5.2. Non-rectangular decision boundary (Left). Decision tree performed on a non-rectangular region (Right)

Figure 5.2 (Right) shows the *approximate division* performed by a simple decision tree.

3. Construction of Decision Trees

- A decision tree is induced from examples. We have a set of labeled patterns which forms the training set. The tree is constructed so that it agrees with the training set.
- The decision tree corresponding to a training set helps us to describe a large number of cases in a concise way. It is an example of *inductive learning*.
- To construct the decision tree, one or more attributes have to be chosen at each node to make a decision.
- The *most important attribute* is used at the earliest level in the decision tree.
- At each node, the set of examples is split up and each outcome is a new decision tree learning problem by itself.
- A really good attribute divides the examples into sets that are all positive or negative.

Example

In figure 5.3, the decision $f_1 \geq a$ divides both class 1 and class 2 so that all the patterns of each class are on one side of the boundary. The decision $f_2 \geq b$ divides the patterns of both Class 1 and Class 2 so that there are two patterns on the other side.

So the decision $f_1 \geq a$ is a better option as it directly classifies the pattern as belonging to Class 1 and Class 2.

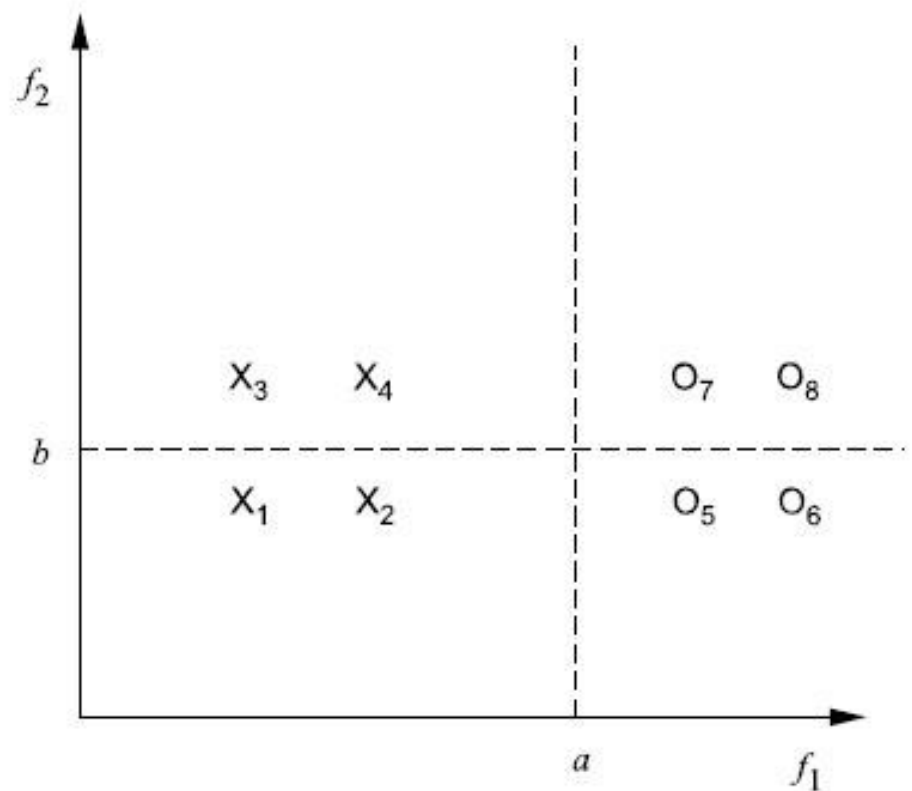


Figure 5.3 Classification into two classes

Once we have chosen the correct attribute, the different outcomes represent new decision trees and, in each case, the *most important attribute* is chosen. This is done until all nodes give a final classification.

Measures of Impurity

- At each node, the decision which makes data at the subsequent nodes as pure as possible is chosen.
- Instead of measuring how pure the node is, we minimize the impurity of the resulting nodes. There are three measures of impurity.
- 1. *Entropy impurity or information impurity*
 - The entropy impurity at a node N is $i(N)$ and is given by

$$i(N) = -\sum_j P(w_j) \log_2 P(w_j)$$

Here $P(w_j)$ is the fraction of patterns at node N of category w_j .

Example: Consider 10 examples which split into three classes. The first class gets 4 examples, the second class gets 5 and the third class gets 1 example.

$$P(w_1) = 4/10 = 0.4; P(w_2) = 5/10 = 0.5; P(w_3) = 1/10 = 0.1$$

And entropy $i(N) = -0.4\log 0.4 - 0.5\log 0.5 - 0.1\log 0.1 = \mathbf{1.36}$.

■ 2. Variance impurity (Gini index)

- $i(N) = P(w_1)P(w_2)$ in a two-category case.
- When $P(w_1)$ is 0.5 and $P(w_2)$ is 0.5, $i(N) = 0.25$. When $P(w_1)$ is 1.0 and $P(w_2)$ is 0, $i(N) = \mathbf{0}$.
- Generalizing this to more categories, we get the Gini impurity

$$i(N) = \sum_{i \neq j} P(w_i)P(w_j) = (1/2)[1 - \sum_j (P(w_j))^2]$$

Example: Consider 10 examples which split into three classes. The first class gets 4 examples, the second class gets 5 and the third class gets 1 example.

$$P(w_1) = 4/10 = 0.4; P(w_2) = 5/10 = 0.5; P(w_3) = 1/10 = 0.1$$

$$\text{And } i(N) = (1/2)[1 - 0.4^2 - 0.5^2 - 0.1^2] = \mathbf{0.29}$$

3. Misclassification impurity

$$i(N) = 1 - \max_j P(w_j)$$

In the example we have considered with three branches, the misclassification impurity will be: $i(N) = 1 - \max(0.4, 0.5, 0.1) = \mathbf{0.5}$

Which attributes to choose?

- The attribute to be chosen should decrease the impurity as much as possible. The *drop in impurity* is defined as:

$$\Delta i(N) = i(N) - P_L * i(N_L) - (1 - P_L) * i(N_R)$$

- This is in the case when there are only two branches at the node. Here P_L and N_L correspond to left branch and N_R corresponds to the right branch. If there are more than two branches, we get

$$\Delta i(N) = i(N) - \sum_j P_j \times i(N_j)$$

j takes on the value for each of the outcomes of the decision made at the node.

That attribute which **maximizes** $\Delta i(N)$ is to be chosen.

Example

- Consider a case where there are 100 examples with 40 belonging to C_1 , 30 belonging to C_2 , and 30 belonging to C_3 . Let some attribute X split these examples into two branches, the left branch containing 60 examples and the right branch containing 40 examples. The left branch contains 40 examples of C_1 , 10 examples of C_2 and 10 examples of C_3 . The right branch contains 0 examples of C_1 , 20 examples of C_2 and 20 examples of C_3 .

$X = a$	$X = b$	Total	Class
Left branch	right branch		

40	0	40	1
10	20	30	2
10	20	30	3

■ Using entropy impurity

$$i(N) = - (40/100)\log(40/100) - (30/100)\log(30/100) - (30/100)\log(30/100) = -0.4\log 0.4 - 0.3\log 0.3 - 0.3\log 0.3 = \mathbf{1.38}$$

The entropy of the left branch $i(N_L)$ is

$$i(N_L) = - (40/60)\log(40/60) - (10/60)\log(10/60) = 1.25$$

The entropy of the right branch $i(N_R)$ is

$$i(N_R) = - (20/60)\log(20/60) - (20/60)\log(20/60) = 1.0$$

The drop in impurity is:

$$\Delta i(N) = 1.38 - ((60/100)*1.25 - ((40/100)*1.0 = 1.38 - 0.75 - 0.4 = \mathbf{0.23}$$

■ Using variance impurity

The impurity at the node is

$$i(N) = (1/2)(1 - 0.4^2 - 0.3^2 - 0.3^2) = 0.5*0.66 = \mathbf{0.33}$$

The impurity at the left node is

$$i(N_L) = (1/2)(1 - 0.667^2 - 0.1667^2 - 0.1667^2) = 0.5*0.5 = 0.25$$

The impurity at the right node is

$$i(N_R) = (1/2)(1 - 0.5^2 - 0.5^2) = 0.5*0.5 = 0.25$$

The drop in impurity is:

$$\Delta i(N) = 0.33 - (60/100)*0.25 - (40/100)*0.25 = \mathbf{0.08}$$

■ Using misclassification impurity

The impurity at the node is

$$i(N) = 1 - \max(0.4, 0.3, 0.3) = 1 - 0.4 = \mathbf{0.6}$$

The impurity at the left node is

$$i(N_L) = 1 - \max(0.667, 0.1667, 0.1667) = 0.333$$

The impurity at the right node is

$$i(N_R) = 1 - \max(0.5, 0.5) = 1 - 0.5 = 0.5$$

The drop in impurity is:

$$\underline{\Delta}i(N) = 0.6 - (60/100) \cdot 0.33 - (40/100) \cdot 0.5 = \mathbf{0.202}$$

The three above attribute selection measures are commonly used for building decision trees.

Besides, there are some other measures, for example, the measure that is based on the *Minimum Description Length* (MDL) principle.

4. Splitting at the nodes

- Each decision outcome at a node is called a *split* since the training data is split into subsets.
- The root node splits the full training data. Each successive decision splits the data at the node into proper subsets.
- The decision rule at each non-leaf node is of the form $f_i(x) = a_0$.
- Depending on this split rule, we can have different types of splitting: *Axis-parallel split*, and *Olique split*.
- Example: Given a set of patterns:
 $X_1 = (1, 1)$; $X_2 = (2, 1)$; $X_3 = (1, 2)$; $X_4 = (2, 2)$; $O_5 = (6, 1)$; $O_6 = (7, 1)$; $O_8 = (7, 2)$; $X_9 = (6, 7)$; $X_{10} = (7, 7)$

Axis-parallel split

The rule at the first node is $f_1 > 4$ and the rule at the node at the next level would be $f_2 > 4$.

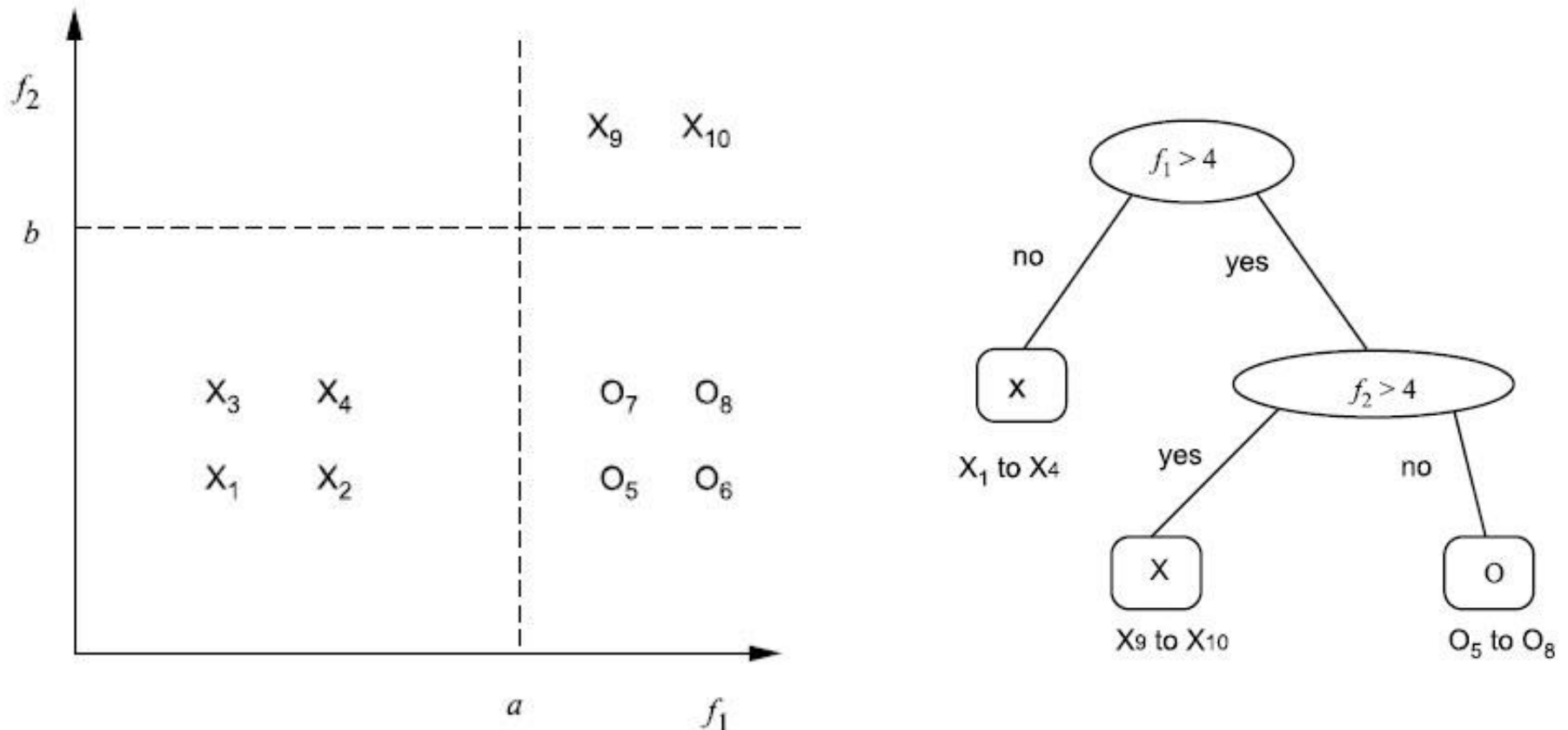


Figure 5.4 Axis-parallel split and the final decision tree.

Olique split

In the case of *olique split* with the general form of equations: $a.f_1 + b.f_2 + c > 0$, we get the following equations: $2a + b + c > 0$ (because of X_2); $7a + 7b + c > 0$ (because of X_{10}) and $6a + 2b + c < 0$ (because of O_7).

Solving these inequalities, we get $a = -1$, $b = 1$ and $c = 2$. The rule in olique split is $-f_1 + f_2 + 2 > 0$ or $f_1 - f_2 < 2$.

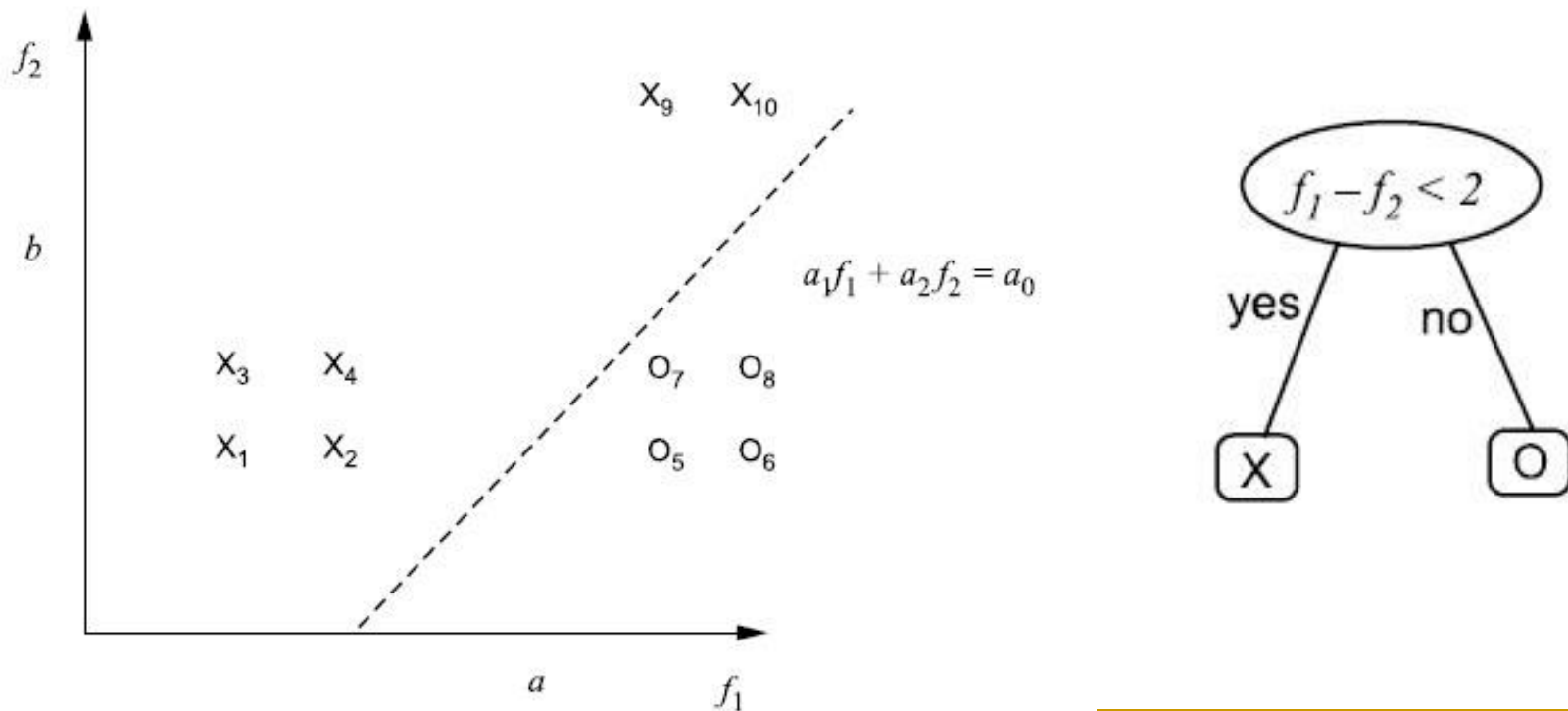


Fig. 5.5 A two-class example of olique split and the decision tree using olique split

Example of decision tree induction

- Suppose we have a data set with 12 patterns as follows

Cook	Mood	Cuisine	Tasty
------	------	---------	-------

Sita	Bad	Indian	Yes
Sita	Good	Continental	Yes
Asha	Bad	Indian	No
Asha	Good	Indian	Yes
Usha	Bad	Indian	Yes
Usha	Bad	Continental	No
Asha	Bad	Continental	No
Asha	Good	Continental	Yes
Usha	Good	Indian	Yes
Usha	Good	Continental	No
Sita	Good	Indian	Yes
Sita	Bad	Continental	Yes

- In this data, there are two classes, Tasty = yes and Tasty = no. Eight examples correspond to Tasty = yes and 4 examples to Tasty = no. The information of the data set is:

$$i(N) = - (4/12)\log(4/12) - (8/12)\log(8/12) = \mathbf{0.918}$$

- Let consider the three attributes and find the attribute with the highest gain.
- 1. Cook
 - (a) Cook = Sita has 4 examples with Tasty = yes and 0 examples with Tasty = no. This branch has an entropy of 0.
 - (b) Cook = Asha has 2 examples with Tasty = yes and 2 examples with Tasty = no. The entropy for Cook = Asha is
$$i(N_A) = -(2/4)\log(2/4) - (2/4)\log(2/4) = 1.0$$
 - (c) Cook = Usha has 2 examples with Tasty = yes and 2 examples with Tasty = no. The entropy for Cook = Usha is
$$i(N_U) = -(2/4)\log(2/4) - (2/4)\log(2/4) = 1.0$$

The gain is

$$\text{Gain(Cook)} = 0.918 - (4/12)*1.0 - (4/12)*1.0 = \mathbf{0.251}$$

■ 2. Mood

- (a) Mood = bad has 3 examples with Tasty = yes and 3 examples with Tasty = no. The entropy for Mood = bad is:

$$i(N_B) = -(3/6)\log(3/6) - (3/6)\log(3/6) = 1.0$$

- (b) Mood = good has 5 examples with Tasty = yes and 1 example with Tasty = no. The entropy for Mood = good is

$$i(N_G) = -(1/6)\log(1/6) - (5/6)\log(5/6) = 0.65$$

The gain is

$$\text{Gain}(\text{Mood}) = 0.918 - (6/12)*0.65 - (6/12)*1.0 = \mathbf{0.093}$$

■ 3. Cuisine

- (a) Cuisine = Indian has 5 examples with Tasty = yes and 1 examples with Tasty = no. The entropy for Cuisine = Indian is:

$$i(N_I) = -(1/6)\log(1/6) - (5/6)\log(5/6) = 0.65$$

- (a) Cuisine = Continental has 3 examples with Tasty = yes and 3 examples with Tasty = no. The entropy for Cuisine = Continental is:

$$i(N_C) = -(3/6)\log(3/6) - (3/6)\log(3/6) = 1.0$$

The gain is:

$$\text{Gain}(\text{Cuisine}) = 0.918 - (6/12)*0.65 - (6/12)*1.0 = \mathbf{0.093}$$

- The attribute with the largest gain is Cook and **Cook** is chosen as the first attribute in the decision tree.
- 1. Cook = Sita
 - Cook = Sita has 4 examples with Tasty = yes and 0 example for Tasty = no, giving an entropy = 0. This branch has reached the leaf node.
- 2. Cook = Asha

Cook = Asha has 2 examples with Tasty = yes and 2 examples with Tasty = no. The entropy for Cook = Asha is:

$$i(N) = -(2/4)\log(2/4) - (2/4)\log(2/4) = 1.0$$

(a) Mood

Mood = bad has 0 examples with Tasty = yes and 2 examples with Tasty = no, giving an entropy of 0. Mood = good has 2 examples with Tasty = yes and 0 examples with Tasty = no, giving an entropy of 0.

The gain for Mood will be $1.0 - 0 - 0 = 1$.

(b) Cuisine

Cuisine = Indian has 1 example with Tasty = yes and 1 example with Tasty = no. The entropy for Cuisine = Indian will be:

$$i(N) = -(1/2)\log(1/2) - (1/2)\log(1/2) = 1.$$

Cuisine = Continental has 1 example with Tasty = yes and 1 example with Tasty = no. The entropy for Cuisine = Continental will be:

$$i(N) = -(1/2)\log(1/2) - (1/2)\log(1/2) = 1.$$

The gain for Cuisine is

$$\text{Gain}(\text{Cuisine}) = 1.0 - (2/4) * 1.0 - (2/4) * 1 = 0$$

Since Mood has a higher gain, it is chosen as the next attribute after Cook = Asha.

■ 3. Cook = Usha

- If we follow the same process, the attribute with the highest gain is Cuisine and Cuisine is the next attribute chosen after Cook = Usha.

The final decision tree is as shown in Figure 5.6. It can be seen that if Cook = Sita, we do not need information about *mood* or *cuisine* to get the right classification. If Cook = Asha, information about the type of *cuisine* is redundant and for Cook = Usha, information about the *mood* is redundant. The leaf nodes represent the class labels.

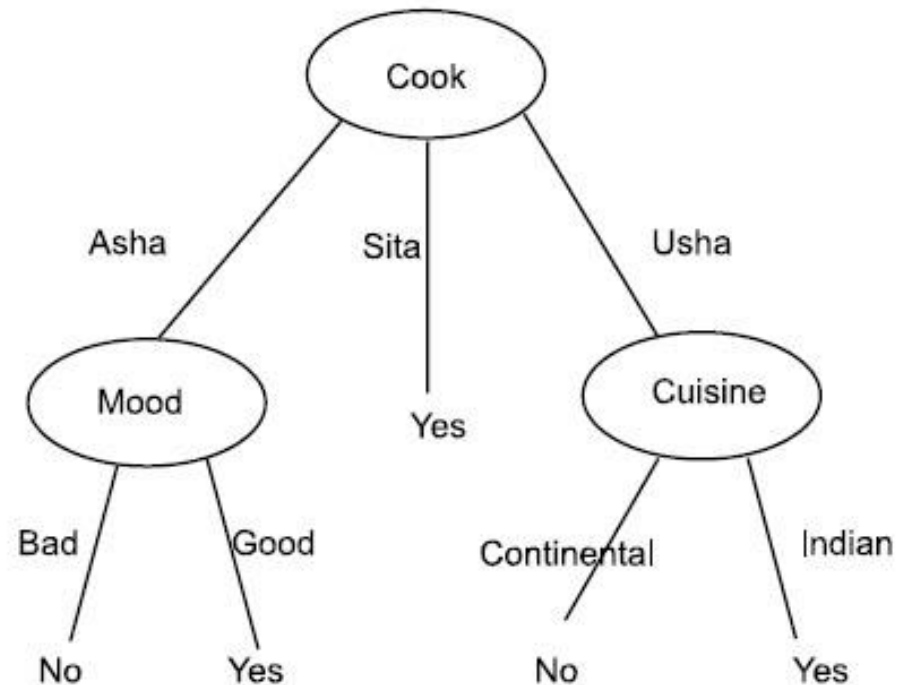


Figure 5.6 Decision tree

Procedure Build_tree(*Records*, *Attributes*);

Begin

- (1) Create a node N ;
- (2) **If** all *Records* belong to the same class, C **then**
- (3) Return N as a leaf node with the class label C ;
- (4) **If** *Attributes* is empty **then**
- (5) Return N as a leaf node with the class label C , such that the majority of *Records* belong to it;
- (6) select attributes A_i (the splitting attribute) from *Attributes*;
- (7) label node N with A_i ;
- (8) **for** each known value a_j of A_i **do**
 begin
- (9) add a branch for node N for the condition $A_i = a_j$;
- (10) S_j = subset of *Records* where $A_i = a_j$;
- (11) **If** S_j is empty **then**
- (12) Add a leaf L with class label C , such that the majority of *Records* belong to it and return L
 else
- (13) Add the node return by Build_tree(S_j , *Attributes* – A_i);
 end
- end**

The algorithm to find the splitting attribute

1. Calculate the entropy of the training set T , using the fraction of the patterns belonging to each class:

$$i(T) = - \sum_j P(w_j) \log_2 P(w_j)$$

$P(w_j)$ is the fraction of patterns of category w_j in the set T .

2. For each attribute, at , that divides T to subset T_i with the fractions P_i , do the following:
 - (i) calculate the entropy $i(T_i)$ of each subset T_i ;
 - (ii) calculate the entropy of attribute at : $i(at) = \sum_i P_i \cdot i(T_i)$
 - (iii) calculate the drop in impurity at attribute at : $\Delta i(at) = i(T) - i(at)$
3. Choose the attribute with the highest value of the drop in impurity.

5. Overfitting and pruning

- The training and test errors are large when the tree is very small. This situation is known as **underfitting**: the model has not yet learned the true structure of the data.
- As the number of nodes in the tree increases, it will have fewer training and test errors.
- However, once the tree becomes large, its test error rate begins to increase even though its training error rate continues to fall. This is known as **overfitting**, when the tree contains some nodes that accidentally fit noisy points in the training set but do not generalize to the test data.
- One way to decide when to stop splitting is by using **validation**. In validation, the tree is trained using a subset of the data (e.g., 90%) with the remaining (10%) kept at a validation set. We continue splitting until the error in the validation set is minimized.
- Another method is to stop splitting when the *reduction in impurity* falls below a certain *threshold* ($\max_s \Delta i(s) \leq \beta$), or when a node represents less than a certain number of points (e.g. 5% of total training set).

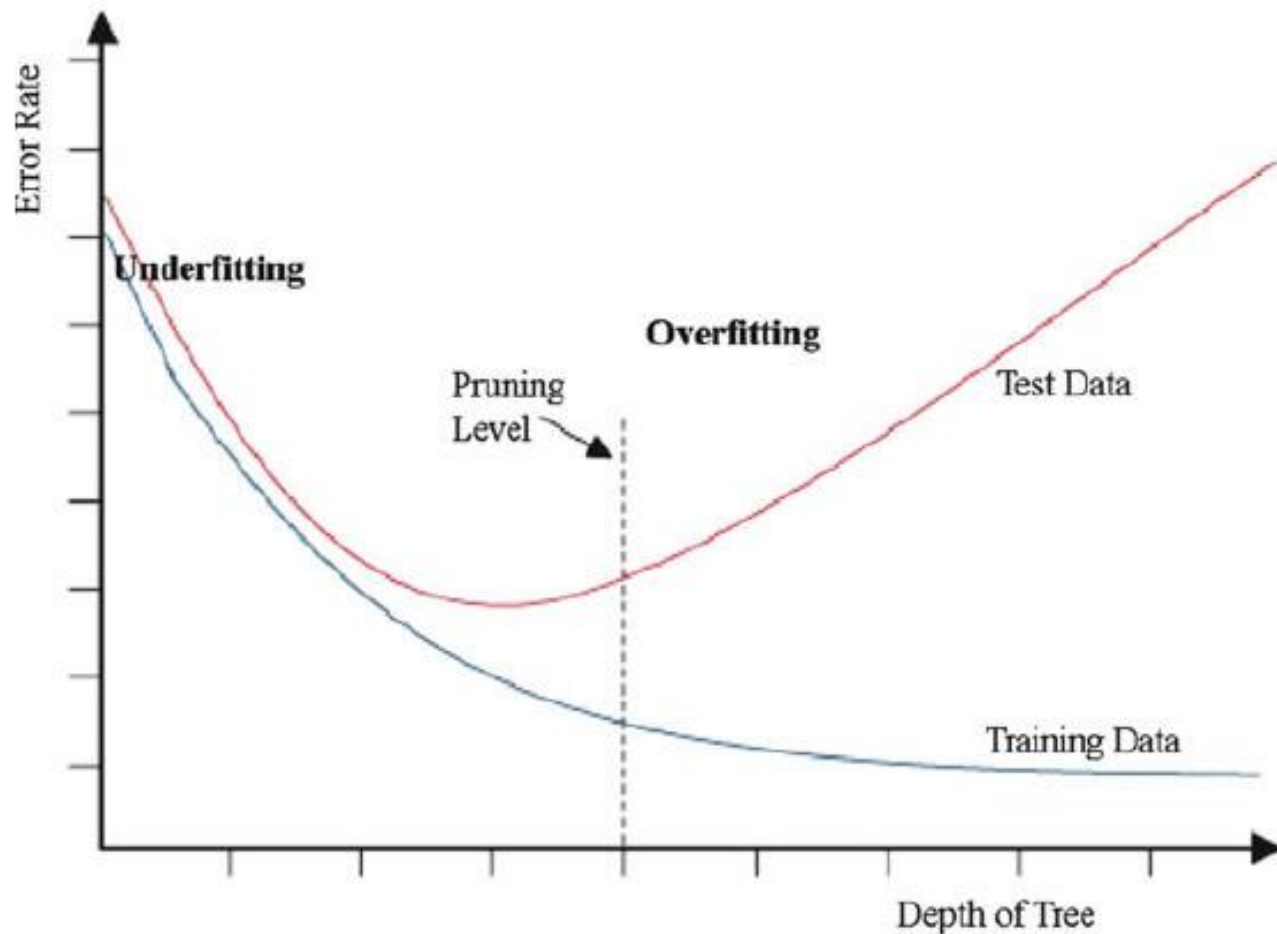


Figure 5.7 Training and test error rates: a typical example. To avoid overfitting, the tree can be post-pruned.

Pruning

- If the tree is grown too large, it can be *pruned* by trimming in a bottom-up fashion.
- All pairs of neighboring leaf nodes (i.e. ones linked to a common parent) are considered for elimination. Any pair whose elimination results in a satisfactory (small) increase in impurity is eliminated, and the common parent node becomes a leaf node. (This node could then be pruned itself.)
- Pruning is incorporated in a successor to the **ID3** algorithm (Quinlan, 1993) known as **C4.5** and in a modification of that, the **J48** algorithm (WEKA software).
- Note: Pruning is computationally expensive.

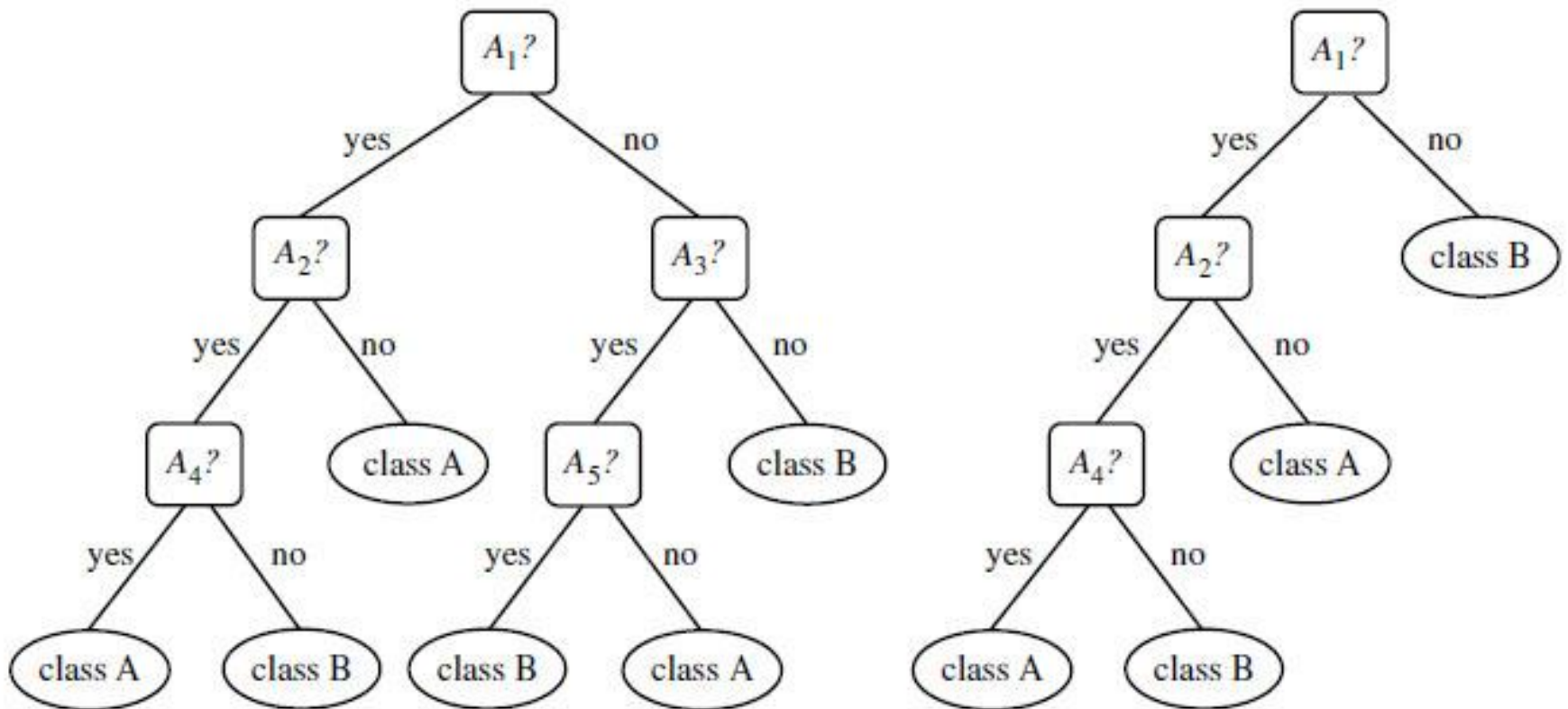


Figure 5.8 An unpruned decision tree and a pruned version of it.

Tools for Decision Tree Classifier

- Decision Tree Classifier is available in:
 - ❑ MATLAB
 - ❑ The data mining software Weka
 - ❑ RapidMiner

References

- M. N. Murty and V. S. Devi, 2011, *Pattern Recognition – An Algorithmic Approach*, Springer.
- G. Dougerty, 2013, *Pattern Recognition and Classification – An Introduction*, Springer.
- J. Han and M. Kamber, *Data Mining: Concepts and Techniques*, 2nd Edition, Morgan Kaufmann Publishers, 2006